UQAC
Département d'informatique
et de mathématique
Université du Québec à Chicoutimi

8INF878 – Intelligence artificielle agentique : Travail No. 1

AUSSI V2 – Résolution de problème et généralisation

Etudiants :
CAPELO DO NOVO Thomas
LOUET Noah
FOUGERAY Mattéo
..

16 octobre 2026

Table des matières

1. Introduction 2
2. Architecture générale 2
   2.1. Organisation du code 2
   2.2. Formulation d'un problème 2
   2.3. Moteur de recherche 2
3. Labyrinthe 3
   3.1. Modélisation 3
   3.2. Heuristique 3
   3.3. Comparaison des algorithmes 3
4. N-Puzzle 4
   4.1. Modélisation 4
   4.2. Heuristique 5
   4.3. Comparaison des algorithmes 5
5. Jeu arithmétique 6
   5.1. Modélisation 6
   5.2. Heuristique 6
   5.3. Comparaison des algorithmes 7
6. Navigation 8
   6.1. Modélisation 8
   6.2. Heuristique 8
   6.3. Comparaison des algorithmes 8
7. Conclusion 8

1. Introduction

Ce travail est réalisé dans le cadre du cours 8INF878 – Intelligence artificielle agentique. Son objectif est de concevoir AUSSI V2 (*Agent Universel de Support Super Intelligent*), un agent capable de résoudre des problèmes grâce à un moteur de résolution générique. Les algorithmes d'exploration ne changent jamais d'un problème à l'autre : seules la formulation du problème et, lorsque c'est nécessaire, son heuristique lui sont propres.

AUSSI résout quatre problèmes : le labyrinthe, la navigation entre villes, le N-Puzzle et le jeu arithmétique. Il dispose de quatre stratégies de recherche (BFS, UCS, Greedy et A*), chacune utilisable en mode Tree Search ou Graph Search. Pour chaque résolution, il indique si une solution a été trouvée, la présente, et fournit son coût, sa profondeur ainsi que des statistiques d'exécution.

2. Architecture générale

2.1. Organisation du code

Le code est découpé en trois parties indépendantes. Ce découpage permet d'ajouter un problème ou une stratégie en modifiant peu de fichiers.

Élément	Rôle	Contenu
`src/`	Moteur générique	Interface `Problem`, stratégies de recherche, fonction `SEARCH`, boucle de l'agent `AGENT`. Cette partie ne connaît aucun problème en particulier.
`problems/`	Problèmes	Une classe par problème (labyrinthe, N-Puzzle, jeu arithmétique, navigation), ses instances de test et un registre qui permet de les construire à partir de leur nom.
`interface/`	Interface utilisateur	Interface graphique pygame : choix du problème, de l'instance, de la stratégie et du mode, puis affichage du résultat.
`main.py`	Point d'entrée	Lance l'interface graphique, ou un affichage en console.

2.2. Formulation d'un problème

Tous les problèmes héritent de la classe abstraite `Problem`, qui est la seule chose que connaît le moteur de recherche. Elle impose de définir :

- `actions(s)` : les actions légales depuis l'état s ;
- `result(s, a)` : l'état obtenu en appliquant l'action a ;
- `goal_test(s)` : vrai si s est un état but ;
- `step_cost(s, a, s')` : le coût de l'action, égal à 1 par défaut ;
- `heuristic(s)` : l'estimation $h(s)$ du coût restant jusqu'au but.

L'état initial est donné à la construction du problème. Un état doit être hashable car le Graph Search l'utilise comme clé de dictionnaire.

2.3. Moteur de recherche

La fonction `SEARCH(problem, strategy, mode)` implémente toutes les stratégies avec une seule boucle. Les nœuds, qui contiennent un état, leur parent, l'action qui y mène, le coût g du chemin et la profondeur, sont stockés dans OPEN, une file de priorité. À chaque itération, AUSSI retire de OPEN le nœud de plus petite valeur $f(n)$, vérifie s'il s'agit du but, puis génère ses successeurs. Seule la fonction f change d'une stratégie à l'autre :

Stratégie	Priorité $f(n)$
BFS	profondeur du nœud

UCS	$g(n)$
Greedy	$h(n)$
A*	$g(n) + h(n)$

En cas d'égalité, le nœud généré en premier est développé en premier. Le test du but est effectué lorsque le nœud est retiré de OPEN (et non lors de sa génération), ce qui garantit l'optimalité d'UCS et de A* avec une heuristique admissible.

Le paramètre `mode` choisit entre les deux variantes d'une même boucle :

- **Tree Search** : aucun état n'est mémorisé. Un même état peut donc être généré plusieurs fois, et indéfiniment s'il existe des cycles.
- **Graph Search** : un dictionnaire retient, pour chaque état, le meilleur coût g connu. Un état déjà connu avec un coût inférieur ou égal n'est pas ajouté à OPEN, et un chemin plus économique vers un état déjà rencontré remplace l'ancien.

Enfin, trois limites de sécurité (nombre de nœuds expansés, taille de OPEN et temps) permettent à AUSSI de s'arrêter lui-même lorsqu'une résolution risque de dépasser ses capacités. `SEARCH` retourne alors l'un des trois statuts suivants : *solution* (un chemin a été trouvé), *aucune solution* (tout l'espace d'états a été exploré) ou *limite atteinte*. Ce dernier cas ne prouve pas l'absence de solution : la recherche a simplement été interrompue.

3. Labyrinthe

3.1. Modélisation

Le problème du labyrinthe consiste à trouver un chemin entre un point de départ et un point d'arrivée dans un labyrinthe représenté par une grille. Chaque cellule de la grille peut être soit libre, soit occupée par un mur, soit un obstacle de poids compris entre 2 et 9. Le but est de se déplacer à travers les cellules pour atteindre l'objectif.

Un état est représenté par la position actuelle de l'agent dans le labyrinthe. À chaque étape, l'agent peut se déplacer vers une cellule adjacente (haut, bas, gauche, droite) si cette cellule est libre ou si elle contient un obstacle.

Le coût du déplacement est de 1 pour une cellule libre et égal au poids de l'obstacle pour une cellule occupée par un obstacle.

3.2. Heuristique

Pour l'heuristique, nous avons choisi d'utiliser la **distance de Manhattan**. Cette heuristique est admissible car elle ne surestime jamais le coût réel pour atteindre l'objectif.

$$h(s) = |x_a - x_g| + |y_a - y_g|$$

Avec $(x_a,\ y_a)$ représentant les coordonnées de l'agent AUSSI et $(x_g,\ y_g)$ représentant les coordonnées de l'objectif. Puisque le coût minimal d'une étape est de 1 (case libre) et que les obstacles ont des poids ≥ 2, la distance de Manhattan calculée avec un coût unitaire reste strictement inférieure ou égale au coût réel ($h(s) \leq h^*(s)$).

3.3. Comparaison des algorithmes

Instance 1 : Facile

Algorithme	Mode	Coût total	Profondeur	Nœuds générés	Nœuds expansés	Max OPEN	Temps (ms)
BFS	Tree	12	12	32 507	14 901	17 606	37.0

BFS	Graph	12	12	37	34	3	0.1
UCS	Tree	12	12	32 507	14 901	17 606	35.1
UCS	Graph	12	12	37	34	3	0.1
Greedy	Tree	12	12	26	12	14	< 0.1
Greedy	Graph	12	12	15	12	3	< 0.1
A*	Tree	12	12	26	12	14	< 0.1
A*	Graph	12	12	15	12	3	< 0.1

Instance 2 : Avec un piège sur une des cases du chemin optimal

Algorithme	Mode	Coût total	Profondeur	Nœuds générés	Nœuds expansés	Max OPEN	Temps (ms)
BFS	Tree	20	12	32 507	14 901	17 606	35.9
BFS	Graph	20	12	37	34	3	0.1
UCS	Tree	N/A	N/A	209 424	100 000	109 424	327.3
UCS	Graph	16	16	41	39	4	0.1
Greedy	Tree	20	12	26	12	14	0.1
Greedy	Graph	20	12	15	12	3	< 0.1
A*	Tree	16	16	154	66	88	0.1
A*	Graph	16	16	22	18	4	< 0.1

Ces résultats mettent en évidence l'efficacité de la distance de Manhattan pour guider la recherche dans le labyrinthe, tout en soulignant l'impact du choix du mode de recherche et de la prise en compte du coût des actions.

Le mode Tree Search montre très rapidement ses limites sur une grille pourtant simple. En raison de la présence de boucles et d'allers-retours possibles entre les cellules adjacentes, les algorithmes sans mémorisation des états visités subissent une explosion combinatoire. Sur l'instance avec piège, UCS et BFS en Tree Search atteignent la limite fixée à 100 000 nœuds expansés ou génèrent des dizaines de milliers de nœuds inutiles. À l'inverse, le passage en Graph Search élimine immédiatement toutes les redondances : le nombre de nœuds expansés chute drastiquement.

L'instance avec un piège illustre le comportement de chaque algorithme face à des coûts non uniformes. BFS et Greedy trouvent une solution avec un coût sous-optimal car ils franchissent l'obstacle pénalisant. En revanche, UCS et A^* parviennent à contourner le piège pour garantir la solution optimale. A^* s'impose comme le choix le plus performant, réussissant à trouver ce chemin optimal en expansant deux fois moins de nœuds que UCS.

4. N-Puzzle

4.1. Modélisation

Le N-Puzzle (aussi connu sous le nom de « Taquin ») est un petit jeu où le but est de recréer une image fractionnée et mélangée en plusieurs cases. La plupart des taquins vendus sont de taille 3x3 mais il est possible d'augmenter la taille comme bon vous semble. Ici, nous testerons AUSSI sur 3 instances : 3x3, 4x4 et 5x5.

Un état est un plateau de tuiles représenté par un tuple de tuples de nombres, représentant chacun la position de leur tuile. Cette représentation très visuelle permet de comparer avec le plateau goal assez facilement.

A chaque étape, AUSSI a le choix de déplacer la case vide vers le haut, le bas, à gauche ou à droite (si possible), et de déplacer la tuile concernée. Le but étant de retrouver le plateau goal.

Chaque déplacement a un coût de 1.

4.2. Heuristique

Pour l'heuristique, nous avons décidé d'utiliser la **distance de Manhattan**. L'heuristique estime le nombre de déplacement qu'il reste avant d'atteindre l'état goal, en calculant pour chaque tuile la distance de Manhattan entre sa position actuelle et sa position dans l'état goal.

$$h(s) = \sum_{t \in T} \left(|r_{s(t)} - r_{g(t)}| + |c_{s(t)} - c_{g(t)}| \right)$$

où T représente l'ensemble des tuiles, r_s et c_s la position actuelle de la tuile et, r_g et c_g la position goal de la tuile

4.3. Comparaison des algorithmes

Instance 3x3

Algorithme	Mode	Coût total	Profondeur	Nœuds générés	Nœuds expansés	Max OPEN	Temps (ms)
BFS	Tree	N/A	N/A	849935	300000*	549935	5036
BFS	Graph	20	20	74203	55408	18858	1273
UCS	Tree	N/A	N/A	849935	300000*	549935	5061
UCS	Graph	20	20	74203	55408	18858	652
Greedy	Tree	N/A	N/A	879586	300000*	579586	7170
Greedy	Graph	62	62	1047	622	413	12
A*	Tree	20	20	12096	4737	7359	75
A*	Graph	20	20	452	282	170	3

* Limite de noeuds expansés (300 000) atteinte, stop par sécurité.

Instance 4x4

Algorithme	Mode	Coût total	Profondeur	Nœuds générés	Nœuds expansés	Max OPEN	Temps (ms)
BFS	Tree	N/A	N/A	4344050	1342381	3001669	30000*
BFS	Graph	N/A	N/A	2891601	1505175	1386426	30000*
UCS	Tree	N/A	N/A	4811051	1486692	3324359	30000*
UCS	Graph	N/A	N/A	2739545	1425505	1314040	30000*
Greedy	Tree	N/A	N/A	2612550	940285	1672265	30000*
Greedy	Graph	109	109	6712	3240	3431	71
A*	Tree	N/A	N/A	2642549	838717	1803832	30000*
A*	Graph	39	39	67117	33965	33069	830

* Limite de temps (30s) atteinte, stop par sécurité.

Instance 5x5

Algorithme	Mode	Coût total	Profondeur	Nœuds générés	Nœuds expansés	Max OPEN	Temps (ms)
BFS	Tree	N/A	N/A	3815403	1097785	2717618	30000*

BFS	Graph	N/A	N/A	2599705	1177149	1422556	30000*
UCS	Tree	N/A	N/A	4404995	1269705	3135290	30000*
UCS	Graph	N/A	N/A	2826946	1282621	1544325	30000*
Greedy	Tree	N/A	N/A	2073682	829472	1244210	30000*
Greedy	Graph	56	56	5733	2472	3199	75
A*	Tree	N/A	N/A	2178969	647533	1531436	30000*
A*	Graph	22	22	26808	11990	14792	364

* Limite de temps (30s) atteinte, stop par sécurité.

Ces résultats nous montrent que l'heuristique joue un rôle clé dans la résolution du taquin. BFS et UCS ne trouvent souvent pas la solution en un temps résonnable, et lors de l'arrêt de la recherche, le nombre de noeuds explorés est énorme. Nous avons essayé de repousser au maximum les limites de temps et de mémoire pour les algorithmes BFS et UCS, mais aucune solution n'était trouvée, alors nous avons laissé les résultats sous 30s par soucis de lisibilité. Plus généralement, sur des instances au-delà de 3x3, il est vite compliqué d'obtenir une solution rapidement avec ces deux algorithmes.

Au contraire, Greedy et A* s'en sortent très bien sur toutes les instances, en tout cas pour leur version Graph Search. On voit que dans ce problème, la version Tree Search a beaucoup plus de difficultés à trouver une solution.

Enfin, on remarque assez bien que BFS et UCS donnent sensiblement les mêmes résultats, ce qui est tout à fait normal car les actions sont ici à coup unitaire.

5. Jeu arithmétique

5.1. Modélisation

Le problème consiste à construire une expression permettant d'atteindre une valeur cible à partir d'un ensemble de nombres et d'opérateurs autorisés.

Un état est représenté par un ensemble de couples (valeur, expression) contenant chaque résultat intermédiaire ainsi que la formule ayant permis de l'obtenir. À chaque étape, deux valeurs sont sélectionnées puis combinées à l'aide d'un opérateur disponible parmi +, -, * ou /.

L'application d'une action remplace les deux opérandes par leur résultat, réduisant ainsi le nombre de valeurs présentes dans l'état. Un état but est atteint lorsqu'il ne reste plus qu'une seule valeur et que celle-ci est égale à la cible recherchée.

Chaque action possède un coût uniforme de 1.

5.2. Heuristique

On peut représenter l'Heuristique utilisée sous la forme :

$$h(s) = (|s| - 1) + \frac{d}{d+1}$$

où :

- $|s| - 1$ représente le nombre minimal d'opérations restantes
- d est l'écart minimal entre une valeur présente dans l'état et la valeur cible

Le premier terme privilégie les états proches d'une expression complète tandis que le second favorise les états contenant une valeur proche de l'objectif.

Cette heuristique n'est pas admissible. Elle a toutefois été retenue afin d'orienter la recherche vers les états les plus prometteurs et de comparer le comportement des algorithmes informés (Greedy et A*) avec celui des algorithmes non informés (BFS et UCS).

5.3. Comparaison des algorithmes

Instance Facile

Algorithme	Mode	Coût total	Profondeur	Nœuds générés	Nœuds expansés	Max OPEN	Temps (ms)
BFS	Tree	3	3	1459	595	1146	0.053s
BFS	Graph	3	3	1459	595	1146	0.069s
UCS	Tree	3	3	1459	595	1146	0.054s
UCS	Graph	3	3	1459	595	1146	0.055s
Greedy	Tree	3	3	53	18	38	0.002s
Greedy	Graph	3	3	53	18	38	0.002s
A*	Tree	3	3	53	6	47	0.002s
A*	Graph	3	3	53	6	47	0.002s

Instance Moyenne

Algorithme	Mode	Coût total	Profondeur	Nœuds générés	Nœuds expansés	Max OPEN	Temps (ms)
BFS	Tree	4	4	58473	16032	45964	2.467s
BFS	Graph	4	4	48897	14112	38308	2.207s
UCS	Tree	4	4	58473	16032	45964	2.397s
UCS	Graph	4	4	48897	14112	38308	2.202s
Greedy	Tree	4	4	579	515	77	0.024s
Greedy	Graph	4	4	579	515	77	0.026s
A*	Tree	4	4	731	187	544	0.029s
A*	Graph	4	4	731	187	544	0.029s

Instance Difficile

Algorithme	Mode	Coût total	Profondeur	Nœuds générés	Nœuds expansés	Max OPEN	Temps (ms)
BFS	Tree	N/A	N/A	265580	19612	245968	10s*
BFS	Graph	N/A	N/A	217432	19312	198120	10s*
UCS	Tree	N/A	N/A	255654	18783	236871	10s*
UCS	Graph	N/A	N/A	194410	17034	177376	10s*
Greedy	Tree	5	5	144	10	136	0.005s
Greedy	Graph	5	5	144	10	136	0.004s
A*	Tree	5	5	458	63	395	0.018s
A*	Graph	5	5	458	63	395	0.020s

* Limite de temps (10s) atteinte, stop par sécurité.

Les résultats montrent clairement que l'heuristique joue un rôle important dans le jeu arithmétique. Les algorithmes non informés, BFS et UCS, voient leur nombre de nœuds générés

augmenter très vite quand la taille du problème grandit. Sur l'instance difficile, aucun des deux ne trouve de solution avant d'atteindre la limite de temps fixée.

À l'inverse, Greedy et A* résolvent toutes les instances testées. Ils explorent aussi beaucoup moins de nœuds, parfois des milliers ou des millions de fois moins. Cette différence vient de l'utilisation de l'heuristique. Elle guide la recherche vers les états les plus prometteurs au lieu d'explorer l'espace d'états de façon uniforme. Bien que cette heuristique ne soit pas admissible, elle donne assez d'informations pour guider la recherche vers une solution. Elle ne garantit pas pour autant que la solution soit optimale.

On remarque aussi que BFS et UCS donnent les mêmes résultats. Cette ressemblance était attendue puisque toutes les actions ont un coût unitaire. Le coût cumulé utilisé par UCS est donc égal à la profondeur utilisée par BFS. Enfin, le mode Graph Search apporte un gain important pour les algorithmes non informés. Il évite d'explorer plusieurs fois les mêmes états, ce qui réduit le nombre de nœuds générés et la taille maximale de la frontière OPEN.

6. Navigation

6.1. Modélisation

6.2. Heuristique

6.3. Comparaison des algorithmes

7. Conclusion